qTask Documentation

[Project & Task Management System]

TABLE OF CONTENTS

1. System Overview
2. Prerequisites
3. Technology Stack
4. User Roles & Permissions
5. Features & Modules
6. How to Run the System
7. Environment Configuration

SYSTEM OVERVIEW

qTask is a role-based Kanban project management system built for software development teams. It supports multi-project workflows with separate Development and QA pipelines, task lifecycle tracking, subtask management, file attachments, activity logging, and analytics.

The system is split into two parts that run concurrently:

- Frontend : React 19 SPA (Vite), served on http://localhost:5173
- Backend : C# .Net 9 Web API with Entity Framework Core + Microsoft Server SQL, served on http:localhost:5216

PREREQUISITES

The following software must be installed before running QTask:

SOFTWARE	VERSION	PURPOSE
Visual Studio Code	>= 1.117	Developer environment for the Frontend
Node.js	>= 18	For npm and Vite + React
Npm (comes with Node.js)	>= 9.x	Package manager (bundled with Node.js)
Visual Studio Community 2022	>= 17.xx.xx	Developer environment for the Backend
SQL Server Management Studio	>= 22	Database server
Git	any	Version control (optional for setup)
Git bash	any	Allows git (cloning project from github

To verify Frontend installations, run:

In cmd:

"node --v"

"npm --v"

TECH STACK

Layer	Technology	Notes
Frontend (JavaScript. HTML, CSS)		
Framework	React	SPA with hooks
Bundler	Vite	Dev server + build
Routing	React Router DOM	URL-based BrowserRouter
Styling	Tailwind CSS (v4)	Utility-first CSS
Charts	ApexCharts	Analytics Charts
Drag & Drop	SortableJS	Kanban drag-and-drop
Icons	Lucide React	SVG icon library
Backend (C#, Microsoft Server SQL)		
Framework	ASP.Net Core	Developer Platform
ORM	Entity Framework	Database Abstraction, OOP Relational Handling
API Platform	Scalar	API Documentation
Database	Microsoft Server SQL	Database
Server Management	SQL Server Management Studio	Easy Database Handling

USER ROLES & PERMISSIONS

Role	**Access Level**	**Description**
Admin	Full	All features. User management, settings, dashboard, projects, analytics, kanban. Can manage severities, statuses, and phases.
ProjectManager	Project-scoped	Can view/manage projects they are assigned as PM. Access to overview, all tasks, and activity log. Must select a project before viewing the kanban board.
Developer	Personal-scoped	Sees only tasks assigned to them as dev assignee. Can filter by their assigned projects via the sidebar. Views My Tasks and All Tasks boards.
QA	Personal-scoped	Sees only tasks assigned to them as QA assignee. Can filter by their assigned projects via the sidebar. Views QA Board and Tasks.

FEATURES & MODULES

5.1 KANBAN BOARD

- Drag-and-drop task movement between columns using SortableJS.
- Columns (phases) are grouped by dev / qa / pm grouping.
- PM sees three separate board sections: Development, QA, Project Management.
- Developer sees only dev-grouped phases. QA sees only qa-grouped phases.
- Moving a task to a "final" phase (isFinal = 1) triggers DoneModal which records the actual end date.
- Cards show: title, severity badge, assignee avatar, progress bar, subtask count, attachment indicator, and status badge.
- Filters: assignee, severity, status — all clearable.
- Collapsible column groups for PM view.

5.2 TASK MANAGEMENT

- Create tasks with: title, description, dev assignee, QA assignee, severity, target date, and initial phase.
- When a project is selected, assignee dropdowns are filtered to only show users assigned to that project (from project_users).
- Edit task fields: dev assignee, QA assignee, severity, status, target date.
- Delete task (with confirmation).
- Task detail modal shows all fields, progress bar, attachments, and subtasks.

5.3 SUBTASKS

- Each task can have multiple subtasks (title + isDone checkbox).
- Subtask view toggled via the "Subtasks" button in TaskDetailModal header — replaces the detail view entirely (no animation, just a clean swap).
- Progress bar (0–100%) is calculated from subtask completion ratio.
- Add, toggle, and delete subtasks in real time.
- Changes saved immediately to the backend
- Server re-inserts all subtasks and returns new DB-assigned IDs; the modal re-syncs local state with server IDs after every save.

5.4 SUBTASK COMMENTS

- Each subtask has a comment icon button in the subtask list.
- Clicking it opens a side panel showing comments for that specific subtask.
- Clicking a different subtask's icon switches the panel to that subtask.
- Comments show commenter avatar (initials), name, date, and text.
- Post comment via input + Post button or Enter key.

5.5 FILE ATTACHMENTS

- Upload files to a task via the FileUpload component in TaskDetailModal.
- Files stored on disk at backend/uploads/{taskId}/.
- Metadata (filename, originalName, mimetype, size) stored in task_attachments.
- Supports preview for image types; download link for other types.
- Delete attachment removes both the DB record and the file on disk.

5.6 PROJECTS

- Admin: full CRUD. Create, edit, delete any project.
- ProjectManager: sees only projects where they are the assigned PM.
- Developer / QA: see only projects they are assigned to via project_users.
- Project form fields: title, description, client name, project manager, status (ongoing / completed / cancelled), target end date, developer assignments (multi-select), QA assignments (multi-select).
- Editing a project replaces project_users (DELETE + re-INSERT).
- KPI cards show: total projects, total tasks, projects with PM, projects with a deadline.

5.7 SIDEBAR PROJECT FILTER (Developer / QA)

- Developers and QA users see a "Projects" section in the sidebar above the navigation links.
- "All Projects" (null) loads all tasks assigned to that user regardless of project.
- Selecting a specific project reloads tasks filtered by that project.
- The section is collapsible. When the sidebar is collapsed, a folder icon shows the active project state; clicking it expands the sidebar.

5.8 DASHBOARD (Admin)

- ❖ KPI summary: total tasks, in-progress tasks, completed tasks, overdue tasks.
- ❖ Recent activity feed.
- ❖ Quick navigation links to Projects and Analytics.

5.9 ANALYTICS (Admin / ProjectManager)

- ❖ Project-level charts powered by ApexCharts / react-apexcharts.
- ❖ Task distribution by phase, severity breakdown, completion trends.
- ❖ Project selector to switch between projects.

5.10 ACTIVITY LOGS

- ❖ All phase changes, task edits, status/severity changes, and are logged to activity_logs.
- ❖ Viewable in the Activity Log page (all roles).
- ❖ Log entries show: action text, task reference, user, and datetime.

5.11 USER MANAGEMENT (Admin only)

- ❖ Create, edit, and deactivate users.
- ❖ Roles: Admin, ProjectManager, Developer, QA.
- ❖ Passwords are bcrypt-hashed before storage.

5.12 SETTINGS (Admin only)

- ❖ Manage severity levels: add, edit label/color, delete, reorder.
- ❖ Manage workflow statuses: add, edit label/color, delete.

HOW TO RUN THE SYSTEM

Frontend Setup (Local Development Setup)

> Github Cloning:

1. Download Node.js (Make sure node is in your system / computer)
2. Create a new folder or use an existing folder
3. Open git bash and navigate to the folder (ex. “cd Desktop/new_folder” or open the folder and terminal as bash in your IDE to show current directory)
4. In the terminal type / bash: “git clone https://github.com/GenD-arc/qtask-kanban” (to get the React Frontend)
5. Once cloned, change directory to qtask-kanban: type “cd qtask-kanban”
6. Install dependencies: type “npm install”
7. Once installed, run the react app (development): type “npm run dev”
8. To allow communication with the backend, configure the environment variables in the folder. Find .env file and change the url to whichever the backend is running on. Ex: VITE_API_URL=”(change this)”

> For production:

1. Make sure the environment base_url is the same as web api url backend
2. Build the React Application: type “npm run build”
3. This command generates a `dist` folder containing minified assets and the final index.html file.
4. This dist folder is going to be used as the frontend

Backend Setup (Local Development Setup)

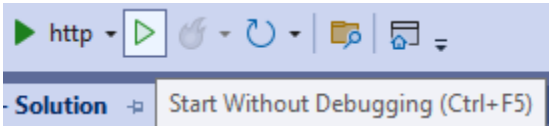
Ideally, the SQL server is set-up first.

> For the SQL Server:

1. Download SQL Server Management Studio 22
2. Open the application.
3. Click on the current options which are
    a. Current Name of the PC\SQLEXPRESS, or
    b. localhost\SQLEXPRESS

> For the API routes:

1. Download Visual Studio Community 2022.
    a. Download the zipped qTask Kanban Solution provided.
    b. Unzip the Solution at any folder location.
2. Opt-in for ASP.Net and Web Development options.
    a. Click on Open a Solution and navigate to the unzipped .sln file.
    b. OR, Open a local folder and navigate to the entire unzipped folder.
3. Navigate to Tools at the top -> NuGet Package Manager for Solution.
    a. Install the following packages:
        - Microsoft.AspNetCore.JsonPatch
        - Microsoft.AspNetCore.OpenAPI
        - Microsoft.EntityFrameworkCore.InMemory
        - Microsoft.EntityFrameworkCore.SqlServer
        - Microsoft.EntityFrameworkCore.Tools
        - Microsoft.VisualStudio.Azure.Cointainers.Tools.Targets
        - Scalar.AspNetCore
4. At the bottom console, find and click on “Package Manager Console”
    a. Type in [Add-Migration Initial] – (without brackets)
    b. Next, type in [update-database]
5. Press Ctrl+F5 or click on to start the backend.



ENVIRONMENT CONFIGURATION

FILE: .env (project root — read by Vite at build time)

VITE_API_URL=http://localhost:5216/api

Used in: src/services/api.js as the BASE_URL for all fetch calls.
Must be prefixed with VITE_ to be exposed to the frontend by Vite.